

项目总结：从零搭建 ROS 2 机器人系统 (Jetson Nano + 麦轮底盘)

一、开发流程

整个项目分为硬件准备、基础系统搭建、软件架构设计、核心功能开发与集成四个阶段：

1. 硬件与基础系统

- 为国产 Jetson Nano 4GB 板卡刷写 JetPack 4.6.5 (Linux for Tegra R32.7.6)，修改设备树 (DTB) 适配 SD 卡检测引脚，实现从 SD 卡启动。
- 关闭图形界面，启用 SSH 与 X11 转发，搭建远程开发环境。
- 在宿主机上创建 Docker 容器 (brain 和 cerebellum)，分别安装 ROS 2 Humble 完整版和基础版，并通过虚拟以太网 (10.10.0.0/16) 实现容器间通信。

2. 软件架构分层

- 采用驱动层 (硬件抽象宏) → 驱动接口层 (dri_interfaces, 发布 ROS 话题) → 模块 (imu_tools、twist_handler、attitude_compensator、inverse_kinematics) → 公共层 (配置加载、日志) 的清晰分层。
- 引入 ros2_control 框架，编写自定义硬件插件 brain_hardware，并集成麦轮控制器。
- 使用 colcon 统一构建，所有参数通过 YAML 配置文件驱动，便于移植。

3. 传感器与底层驱动

- 完成串口通信、CRC 校验，实现控制板指令收发。
- 编写 IMU、BMS、电机驱动接口，输出 /imu/data_raw、/bms/voltage、/chassis/motor_states 等话题。

4. 建图与导航集成

- 集成 slam_toolbox 实现 2D 激光 SLAM，手动建图后保存地图。
- 集成 Nav2 栈 (amcl、planner_server、controller_server、bt_navigator)，实现单点与多点导航。
- 编写多个启动文件：brain_mapping.launch.py (纯建图)、brain_localization.launch.py (纯导航)、brain_slam_nav.launch.py (边建图边导航)。

驱动模块：控制板通信、IMU、BMS、motor，重构完成

驱动接口模块：imu：零漂校准、单位转换、发布四元数 bms：读取原始数据、底盘：订阅/joint_states、发布/chassis/motor_states、看门狗

公共模块：日志、配置加载，重构完成

系统服务：零漂校准

接口：底盘发布消息

启动：一键全部启动

组件：ros2_control

二、调试工具的使用

在整个开发过程中，以下工具是排查问题的关键：

1. rviz2

- 用于可视化机器人模型、激光雷达点云、地图、TF 树、路径规划等。

- 常见问题:

- 机器人模型不显示 → 检查 `Fixed Frame` 是否设置为 `map` 或 `odom`, 确认 `/tf` 话题有数据。
- 地图不显示 → 确认 `/map` 话题有发布者, 且 `Color Scheme` 设置为 `map` 或 `costmap`。
- OpenGL 渲染错误 → 使用 `export LIBGL_ALWAYS_SOFTWARE=1` 启动软件渲染。

2. rqt

- `rqt_graph`: 查看节点和话题拓扑, 验证数据流是否正常。
- `rqt_plot`: 绘制 `/odom/pose/pose/position/x`、`/odom/twist/twist/linear/x`、`/cmd_vel/linear/x` 等曲线, 快速判断里程计是否正常累积、速度指令是否正确执行。
- `rqt_console`: 实时查看各节点的日志输出, 定位错误来源。

3. tf2_tools

- `tf2_echo`: 实时查看两个坐标系之间的变换, 如 `ros2 run tf2_ros tf2_echo map odom`, 用于调试 TF 树断裂或变换超时 (例如 EKF 启动后 `planner_server` 报错找不到 `map` 帧)。
- `view_frames`: 生成 PDF 图形化显示整个 TF 树, 快速定位缺失的坐标帧 (例如发现 `base_link` 没有连接到 `odom`)。
- `tf2_monitor`: 监控 TF 发布频率和延迟, 辅助排查时间戳问题。

三、IMU 坐标变换的曲折历程 (核心难点与解决方案)

3.1 误区: 手动编写轴映射与零漂校准

初始阶段, 我误以为 URDF 不会自动处理传感器坐标变换, 于是编写了一套纯代码映射方案, 将 IMU 物理轴的加速度和角速度转换到 ROS 2 标准坐标系 (ENU)。

- **加速度**: 通过轴映射 (例如 `front: "y"`、`left: "-x"`、`up: "z"`) 并应用 SM 矩阵 `[[0,-1,0],[1,0,0],[0,0,1]]`, 得到正确方向。
- **角速度**: 同样用 SM 矩阵处理, 并减去零漂 (bias) 校准。
- **四元数**: 先由互补滤波计算绝对四元数, 再实现零点校准 (相对四元数), 记录静止姿态作为参考原点。
结果: 加速度和角速度正确, 但四元数的 yaw 符号相反, 即使后续用静态 TF 修正旋转, TF 变换后的四元数仍不符合预期。

3.2 真相: URDF 自动处理坐标变换

后期才了解到, `robot_state_publisher` 会根据 URDF 中 `imu_link` 的 `origin` 定义自动发布 `base_link → imu_link` 的静态 TF, 而 EKF 和 `imu_transformer` 等节点会自动读取该 TF 进行坐标变换。这意味着只要 URDF 中的 IMU 安装姿态 (rpy) 定义正确, 就不需要手动编写任何坐标变换代码。

3.3 最终解决方案

- **驱动层仅发布原始数据**: IMU 驱动 (`imu_dri`) 只负责读取硬件数据, 经过零漂校准 (bias) 后, 直接发布原始的加速度、角速度和由互补滤波计算的四元数, `frame_id` 设为 `imu_link`。
- **URDF 描述安装姿态**: 在 URDF 中准确描述 IMU 的安装位置和朝向 (`<origin xyz="..." rpy="0 0 1.57"/>`), `robot_state_publisher` 自动发布 `base_link → imu_link` 的 TF。
- **EKF 自动转换**: `robot_localization` 节点订阅 `/imu/data_raw` 时, 自动查找 `imu_link` 到 `base_link` 的 TF, 将所有数据转换到 `base_link` 坐标系下进行融合。

- **校准工具保留**：仍保留零漂校准、零点四元数校准等服务，作为驱动层的辅助工具，但不再手动干预坐标变换。

四、ros2_control 的深入学习与自主实现

4.1 初始误区

最初尝试使用官方 `mecanum_drive_controller`，但配置文件一直报错找不到 `ki` 等 PID 参数，折腾数天后放弃。

教训：官方控制器虽然通用，但对 URDF 和参数配置要求严格，且文档晦涩，不如自主实现灵活可控。

4.2 理解 ros2_control 架构

- **组成**：URDF 中的 `<ros2_control>` 描述硬件和关节接口 → **硬件插件**（继承 `SystemInterface`）→ **控制器**（继承 `ControllerInterface`）→ **控制器管理器**（`controller_manager`）统一加载和管理。
- **关键概念**：关节名是硬件插件和控制器之间的**唯一纽带**，接口类型（`velocity`、`position`）必须一致。

4.3 硬件插件实现

继承 `hardware_interface::SystemInterface`，实现完整的生命周期管理：

- `on_init`：初始化关节名列表、硬件通信（串口、CRC）、分配命令/状态接口存储空间。
- `on_configure`：创建 ROS 订阅/发布者（例如订阅 `/chassis/motor_states` 接收硬件反馈）。
- `on_activate`：清零命令/状态缓冲区，启动独立 `spin` 线程处理 ROS 事件。
- `export_state_interfaces` / `export_command_interfaces`：将关节名与私有变量（`hw_states_`、`hw_positions_`、`hw_commands_`）绑定导出。
- `read`：从硬件读取电机状态，更新 `hw_states_` 和 `hw_positions_`。
- `write`：将 `hw_commands_` 中的速度指令发送给电机。
- 使用 `PLUGINLIB_EXPORT_CLASS` 导出插件，并通过 `brain_hardware_plugins.xml` 描述插件库。

4.4 控制器实现

继承 `controller_interface::ControllerInterface`：

- `on_init`：从 `get_node()` 读取 `wheel_radius`、`wheel_separation_h`、`wheel_separation_w`、`max_speed` 等参数。
- `on_configure`：订阅 `/cmd_vel`（使用 `realtime_tools::RealtimeBuffer` 缓存指令），创建 `/odom` 发布者。
- `on_activate`：清零里程计，注册关节句柄（`register_joint_handles`），通过关节名匹配硬件插件的命令/状态接口。
- `command_interface_configuration` / `state_interface_configuration`：声明需要哪些接口（速度命令、速度状态、位置状态）。
- `update`：
 - 从实时缓存读取最新 Twist 指令；
 - 通过逆运动学公式计算四轮速度；
 - 将速度写入命令接口（`velocity_command.get().set_value(...)`）；
 - 从状态接口读取实际速度，进行里程计积分，发布 `/odom` 话题（TF 发布后续删除，以避免与 EKF 冲突）。

- 使用 `PLUGINLIB_EXPORT_CLASS` 导出控制器插件。

4.5 配置与集成

- `brain_controllers.yaml` 中定义控制器管理器、`joint_state_broadcaster` 和 `brain_controller`。
- **关键点：**`joint_state_broadcaster` 必须读取 `position` 状态接口，否则 `rviz2` 显示 TF 为 `nan`。
- `controller_manager` 通过 `ros2_control_node` 启动，加载硬件插件和控制器。

五、EKF、SLAM 与 Nav2 集成中的关键问题

1. 激光雷达坐标系一致性

- 雷达驱动 `frame_id` 必须与 URDF 中 `lidar_frame` 完全一致，否则 TF 找不到。
- 通过 `tf2_echo` 验证 `base_link` → `lidar_frame` 是否存在。

2. EKF 的 yaw 限制问题

- IMU 的 `orientation` 被互补滤波限制在 $\pm 30^\circ$ ，导致旋转无法累积。
- 改为 EKF 仅使用角速度（`imu0_config` 中关掉 `orientation`，启用 `vyaw`），让滤波器自行积分角度，解决回正问题。

3. TF 树断裂

- EKF 启动后 `planner_server` 报错 `Timed out waiting for transform from base_link to map`。
- 用 `tf2_echo` 发现缺少 `map` → `odom`，确认 AMCL 未激活且 `map_server` 需先于 AMCL 启动。
- 通过 `lifecycle_manager` 按序管理：`map_server` → `amcl` → Nav2 其他节点。

4. 地图叠加

- 在导航模式下，`slam_toolbox` 仍发布 `/map` 与 `map_server` 冲突。
- 最终移除 `slam_toolbox`，仅用 `map_server` + AMCL 进行纯定位导航。

5. RViz 渲染问题

- 地图不显示或 OpenGL 报错，用 `export LIBGL_ALWAYS_SOFTWARE=1` 启动 `rviz2` 解决。

六、最终成果

- 完成了一套 **可移植、可配置** 的机器人软件系统，包含：
 - **底层驱动**（串口通信、IMU、BMS、电机控制）
 - **驱动接口层**（发布标准 ROS 消息）
 - **算法层**（IMU 处理、速度限制、姿态补偿、麦轮逆运动学）
 - **ros2_control**（硬件插件 + 控制器管理）
 - **SLAM 建图**（`slam_toolbox`）
 - **自主导航**（Nav2 单点/多点导航）
- 所有参数通过 YAML 配置，换车仅需修改驱动宏和配置文件，无需改动核心代码。
- Docker 容器化开发环境保证了团队协作与跨平台一致性。

整个开发周期约三周，涵盖硬件底层、ROS 中间件、算法、导航与调试，是一次完整的机器人系统从零到一的工程实践。

还有调试工具的使用，rviz2、rqt、tftools也写一下吧。

驱动层最重点的是IMU坐标变换，我一开始不知道URDF会通过静态发布机器人状态自动进行坐标变换，然后自己写了一套方案纯代码把IMU坐标系转换到了ROS2标准坐标系上，加速度和角速度微量没问题了，但是在四元数那就还原回原本IMU安装的位置了，然后把绝对四元数改为相对四元数（同时实现了零点校准+零漂校准功能），但是发现相对四元数的yaw角度符号相反，我再用tf静态发布到ros2坐标的旋转。然后就正常了。然后我觉得只用tf应该也行，于是重构代码，想把加速度和角速度也tf变换过去，结果发现tf2静态变换输出的四元数不改变，于是我找厂商的资料发现一个校准文件，记录了SM矩阵和零漂参数。但是厂商的SM矩阵是单位矩阵，然后我又重构代码，将SM矩阵修改为我的安装位置SM=
[0,-1,0]
[1,0,0]
[0,0,1]

然后SM*原始加速度[x,y,z]=ros2标准的[x,y,z]，SM-(角速度[-y,x,z] - 零漂)=ros2标准[x,y,z]，然后再计算相对四元再发布yaw旋转90°的tf变换，同时提供SM矩阵计算、零漂校准、零点四元数校准、旋转角的校准服务。最后我发现了加载urdf文件会自动处理坐标变换，天都塌了。然后我的驱动层就只发布原始的加速度、角速度、直接计算四元数、发布imu_link坐标系，交给robot_state_publisher处理。

然后就是ros2_control，一开始不知道他的架构就跟着AI瞎搞，说使用mecanum_drive_controller控制器就不需要写控制器代码了，然后配置文件就一直报错都不到ki的参数，折腾了几天之后就放弃了，自己写控制器，然后了解到ros2_control的由urdf ros2_control插件组成、硬件组件、控制器、控制器管理器、硬件管理器组成，然后开始自己写urdf，了解了怎么写，虽然urdf也是借助ai工具做的，

```
<ros2_control name="BrainSystem" type="system">
```

```
  brain_hardware/BrainSystemHardware
```

```
  <command_interface name="velocity"/>
```

```
  <state_interface name="position"/>
```

```
  <state_interface name="velocity"/>
```

```
  <command_interface name="velocity"/>
```

```
  <state_interface name="position"/>
```

```
  <state_interface name="velocity"/>
```

```
  <command_interface name="velocity"/>
```

```
  <state_interface name="position"/>
```

```
  <state_interface name="velocity"/>
```

```
  <command_interface name="velocity"/>
```

```
  <state_interface name="position"/>
```

```
  <state_interface name="velocity"/>
```

```
</ros2_control>
```

下一步是做硬件组件根据urdf ros2_control的类型是System就继承

hardware_interface::SystemInterface类实现了生命周期管理：on_init初始化时，进行硬件初始化、关节名和Id以及command_interface和state_interface接口初始化；on_configure配置时，进行硬件配置和特殊节点发布订阅；on_activate激活时，对command_interface和state_interface接口的

hw_commands、hw_states、hw_positions清零并启用spin线程进行事件循环。on_deactivate销毁时进行节点销毁，然后进行接口导出类型关节名、hardware_interface类型、私有变量——对应，然后导出。

std::vector

```
BrainSystemHardware::export_state_interfaces() {  
    std::vector state_interfaces;  
    for (size_t i = 0; i < joint_names.size(); ++i) {  
        state_interfaces.emplace_back(  
            joint_names[i], hardware_interface::HW_IF_VELOCITY, &hw_states[i]);  
        state_interfaces.emplace_back(  
            joint_names[i], hardware_interface::HW_IF_POSITION, &hw_positions[i]);  
    }  
    return state_interfaces;  
}
```

std::vector

```
BrainSystemHardware::export_command_interfaces() {  
    std::vector command_interfaces;  
    for (size_t i = 0; i < joint_names.size(); ++i) {  
        command_interfaces.emplace_back(  
            joint_names[i], hardware_interface::HW_IF_VELOCITY, &hw_commands[i]);  
    }  
    return command_interfaces;  
}
```

然后实现read操作，将数据读取到hw_states、hw_positions里，也就是导出的接口变量。

然后实现read操作，将hw_commands数据写到电机里，hw_commands也是导出的接口变量。

```
PLUGINLIB_EXPORT_CLASS(  
    brain_hardware::BrainSystemHardware,  
    hardware_interface::SystemInterface)
```

然后导出插件，硬件就完成了，brain_hardware是命名空间、BrainSystemHardware是类，hardware_interface::SystemInterface是父类。这里的命名空间和类名要和urdf里描述的一致。然后进行xml插件导出。

下一步就是做控制器，继承ControllerInterface类，这里对命名空间和类型没有要求吧，

在on_init阶段需要从auto node = get_node();里读取参数，参数写在控制器管理器的配置文件里，并且关节名和urdf一定要对上，因为控制器和硬件组件就是靠关节名进行联系的；on_configure阶段进行/cmd_vel订阅，与/odom发布；on_activate阶段，对各个数据清0初始化；on_deactivate阶段清理数据；然后就是对命令和状态接口进行配置，

// ===== 接口配置 =====

```
controller_interface::InterfaceConfiguration BrainController::command_interface_configuration()  
const {  
    controller_interface::InterfaceConfiguration config;  
    config.type = controller_interface::interface_configuration_type::INDIVIDUAL;  
  
    for (const auto &joint : joint_names_) {  
        config.names.push_back(joint + "/" + hardware_interface::HW_IF_VELOCITY);  
    }  
  
    return config;  
}
```

```
controller_interface::InterfaceConfiguration BrainController::state_interface_configuration() const  
{  
    controller_interface::InterfaceConfiguration config;  
    config.type = controller_interface::interface_configuration_type::INDIVIDUAL;
```

```

for (const auto &joint : joint_names_) {
    config.names.push_back(joint + "/" + hardware_interface::HW_IF_VELOCITY);
}

for (const auto &joint : joint_names_) {
    config.names.push_back(joint + "/" + hardware_interface::HW_IF_POSITION);
}

return config;
}

```

然后注册接口句柄，拿到关节名对应硬件组件的`command_interfaces`、`state_interfaces`变量

```

struct WheelHandle {
    std::reference_wrapper<hardware_interface::LoanedCommandInterface> velocity_command;
    std::reference_wrapper<const hardware_interface::LoanedStateInterface> velocity_state;
    std::reference_wrapper<const hardware_interface::LoanedStateInterface> position_state;
};

std::vector<WheelHandle> wheel_handles;
// ===== 注册句柄 =====
bool BrainController::register_joint_handles() {
    wheel_handles.clear();
    wheel_handles.reserve(joint_names.size());

    for (const auto &joint_name : joint_names) {
        auto cmd_it = std::find_if(
            command_interfaces_.begin(),
            command_interfaces_.end(),
            [&joint_name] {
                return interface.get_prefix_name() == joint_name &&
                    interface.get_interface_name() == hardware_interface::HW_IF_VELOCITY;
            });
    }
}

```

```

if (cmd_it == command_interfaces_.end()) {
    RCLCPP_ERROR(get_node()->get_logger(), "Cannot find velocity command for %s",
        joint_name.c_str());
    return false;
}

auto state_it = std::find_if(
    state_interfaces_.cbegin(),
    state_interfaces_.cend(),
    [&joint_name](const hardware_interface::LoanedStateInterface &interface) {
        return interface.get_prefix_name() == joint_name &&
            interface.get_interface_name() == hardware_interface::HW_IF_VELOCITY;
    });

if (state_it == state_interfaces_.cend()) {
    RCLCPP_ERROR(get_node()->get_logger(), "Cannot find velocity state for %s",
        joint_name.c_str());
    return false;
}

auto pos_it = std::find_if(
    state_interfaces_.cbegin(),
    state_interfaces_.cend(),
    [&joint_name](const hardware_interface::LoanedStateInterface &interface) {

```



```

        return interface.get_prefix_name() == joint_name &&
               interface.get_interface_name() == hardware_interface::HW_IF_POSITION;
    });

    if (pos_it == state_interfaces_.cend()) {
        RCLCPP_ERROR(get_node()->get_logger(), "Cannot find position state for %s",
            joint_name.c_str());
        return false;
    }

    wheel_handles_.push_back({std::ref(*cmd_it), std::ref(*state_it),
        std::ref(*pos_it)});
    RCLCPP_INFO(get_node()->get_logger(), "Registered handle for %s",
        joint_name.c_str());

}

return true;
}

```

然后就是/cmd_vel回调

```

realtime_tools::RealtimeBuffer<geometry_msgs::msg::Twist> velocity_command_buffer;
void BrainController::cmdVelCallback(const geometry_msgs::msg::Twist::SharedPtr msg) {
    velocity_command_buffer.writeFromNonRT(*msg);
}

```

/odom发布与tf发布（这里的tf发布后续删除了，因为我发现tf_tools显示会有多个odom发布，因为后续的ekf也会发布），父坐标为odom、base_link。

最后就是controller_interface::return_type BrainController::update主循环实现

读取Twist消息的实时缓存并通过ik公式计算四轮速度并写入命令接口中。

从状态接口里读取四轮速度，进行里程计计算与发布。最后导出插件

```

PLUGINLIB_EXPORT_CLASS(
    brain_controller::BrainController,
    controller_interface::ControllerInterface
)

```

然后在/ros2_ws/src/description/config/brain_controllers.yaml编写控制器管理的配置，

joint_state_broadcaster/JointStateBroadcaster会进行ros2_control管理的关节发布，发布的都是urdf定义的state接口。state接口在硬件组件处都需要读取，不然rviz2会显示坐标变换发布为nan报错。

controller_manager:

```

ros__parameters:
  update_rate: 50
joint_state_broadcaster:
  type: "joint_state_broadcaster/JointStateBroadcaster"
brain_controller:
  type: "brain_controller/BrainController"

```

joint_state_broadcaster:

```

ros__parameters:
  {}

```

brain_controller:

```

ros__parameters:
  wheel_radius: 0.04
  wheel_separation_h: 0.18

```


wheel_separation_w: 0.173

max_speed: 1.33 #rs

然后就是下载激光雷达驱动，编写slam_params.yaml，然后使用键盘twist工具进行运动，rviz2无法显示，然后修改雷达的启动文件，修改坐标系名称为lidar_frame，和urdf必须对上，然后就是话题、硬件端口的配置，rviz2可以进行显示了但是漂移非常严重，然后进行ekf的引入，然后ekf遇到的问题就是坐标系混乱，ekf提示找不到base_link的变换，用tf2_tools调试看，然后就是和你的对话了，帮我继续说下去然后总结一下。

稍微详细一点

map->odom(ekf)->base_link->各个数据

```
digraph G {
  "base_link" -> "wheel_back_left_link"[label=" Broadcaster:
default_authority\nAverage rate: 36.576\nBuffer length: 5.14\nMost recent
transform: 1784457131.699668\nOldest transform: 1784457126.559673\n"];
  "odom" -> "base_link"[label=" Broadcaster: default_authority\nAverage rate:
30.212\nBuffer length: 4.7\nMost recent transform: 1784457131.670903\nOldest
transform: 1784457126.970793\n"];
  "base_link" -> "wheel_back_right_link"[label=" Broadcaster:
default_authority\nAverage rate: 36.576\nBuffer length: 5.14\nMost recent
transform: 1784457131.699668\nOldest transform: 1784457126.559673\n"];
  "base_link" -> "wheel_front_left_link"[label=" Broadcaster:
default_authority\nAverage rate: 36.576\nBuffer length: 5.14\nMost recent
transform: 1784457131.699668\nOldest transform: 1784457126.559673\n"];
  "base_link" -> "wheel_front_right_link"[label=" Broadcaster:
default_authority\nAverage rate: 36.576\nBuffer length: 5.14\nMost recent
transform: 1784457131.699668\nOldest transform: 1784457126.559673\n"];
  "base_link" -> "base_footprint"[label=" Broadcaster: default_authority\nAverage
rate: 10000.0\nBuffer length: 0.0\nMost recent transform: 0.0\nOldest transform:
0.0\n"];
  "base_link" -> "camera_link0"[label=" Broadcaster: default_authority\nAverage
rate: 10000.0\nBuffer length: 0.0\nMost recent transform: 0.0\nOldest transform:
0.0\n"];
  "base_link" -> "imu_link"[label=" Broadcaster: default_authority\nAverage rate:
10000.0\nBuffer length: 0.0\nMost recent transform: 0.0\nOldest transform:
0.0\n"];
  "base_link" -> "lidar_frame"[label=" Broadcaster: default_authority\nAverage
rate: 10000.0\nBuffer length: 0.0\nMost recent transform: 0.0\nOldest transform:
0.0\n"];
  "lidar_frame" -> "lidar_sim_frame"[label=" Broadcaster:
default_authority\nAverage rate: 10000.0\nBuffer length: 0.0\nMost recent
transform: 0.0\nOldest transform: 0.0\n"];
  "base_link" -> "mic_link"[label=" Broadcaster: default_authority\nAverage rate:
10000.0\nBuffer length: 0.0\nMost recent transform: 0.0\nOldest transform:
0.0\n"];
  "map" -> "odom"[label=" Broadcaster: default_authority\nAverage rate:
10.233\nBuffer length: 3.323\nMost recent transform: 1784457131.733256\nOldest
transform: 1784457128.410513\n"];
  edge [style=invis];
  subgraph cluster_legend { style=bold; color=black; label = "view_frames Result";
  "Recorded at time: 1784457131.7956152"[ shape=plaintext ] ;
  }->"map";
}
```

}

